

UPPSALA UNIVERSITY



ACCELERATOR-BASED PROGRAMMING

Final Project: GPU Optimization of Sparse Matrix-Vector Multiplication

Author:

Abhinav RAMALINGAM

September 15, 2026

1 Introduction

This project focuses on the high-performance acceleration of the Conjugate Gradient (CG) iterative solver, applied to large, sparse linear systems generated by the Finite Element Method (FEM). Given that the Sparse Matrix-Vector Multiplication (SpMVM) is the computational bottleneck of the solver, the core objective is to implement and optimize this operation on an NVIDIA GPU using the CUDA framework. The project contrasts the performance of a baseline Compressed Row Storage (CRS) implementation against a highly optimized, architecture-aware SELL-C- σ format, analyzing the resulting speedups and memory access efficiency gains.

2 Tasks

Task 1: CRS: CPU Implementation and results

The task is to compile the CPU code `fem_application.cpp`. The command is as follows:

```
g++ -O3 -march=sandybridge -fopenmp fem_application.cpp -o app.host
```

A bash script was used to run the program for different problem sizes (N) and precision types (`float` and `double`) and logs all outputs to a file. The export variables control parallelism and optimize resource usage, specifically setting `OMP_NUM_THREADS = 16` and using `OMP_PROC_BIND = spread` for consistent timing. All configurations, regardless of problem size, were executed **200** times via the `-repeat` flag to ensure reliable performance measurement.

The results were as follows:

N	L2 Error	MV Size	MV Time	MV BW	CG Iters	CG Time	CG Error	MUPD/s/it
32	0.039458	32768	0.000794	9.029	99	0.089828	1.92625e-05	36.114
64	0.010180	262144	0.007551	7.822	204	1.761600	9.71706e-05	30.357
128	0.002700	2097152	0.065092	7.367	454	32.3151	0.000324413	29.463
256	0.003103	16777216	0.650490	5.940	500	353.789	0.00344846	23.711

Table 1: CPU Performance and accuracy results for `float` precision at different domain sizes.

N	L2 Error	MV Size	MV Time	MV BW	CG Iters	CG Time	CG Error	MUPD/s/it
32	0.039457	32768	0.001050	10.243	84	0.102660	5.46266e-12	26.812
64	0.010184	262144	0.010666	8.307	142	1.717870	2.82115e-11	21.669
128	0.002587	2097152	0.089857	8.004	234	23.8379	8.88701e-11	20.586
256	0.000652	16777216	0.733888	7.898	341	281.569	7.2164e-10	20.318

Table 2: CPU Performance and accuracy results for `double` precision at different domain sizes.

As we can see in Tables 1 and 2, the L2 Discretization Error reduces by a factor of ≈ 4 when N doubles, which is expected as the FEM solution exhibits second-order convergence ($h^2 \sim (1/N)^2$). The Conjugate Gradient Error is around 10^{-10} to 10^{-12} for **double** data type, and around 10^{-4} to 10^{-5} for float data type. For $N = 256$ in **float** precision, the L2 Error reduction is not as expected, which can likely be attributed to round-off errors accumulating in single-precision arithmetic at large problem sizes (similar reason why the CG error is quite high too). Also, the number of iterations increases with increase in problem size, which is a characteristic feature of solving linear systems using FEM (number of iterations required in CG method is proportional to the condition number ($\sqrt{\kappa}$) of the matrix), which in turn is proportional to N .

Task 2: CRS: CUDA Implementation and results

The CUDA implementation was completed by filling in the *TODOS* in the **vector.hpp** and **sparse_matrix.hpp** files. The core matrix-vector product kernel (compute_spmv) utilizes the standard Thread-Per-Row (TPR) strategy for the CRS format, where one CUDA thread is responsible for computing the dot product for a single row. As specified in the instructions, this task was specifically implemented for $C=32$, and $\sigma=1$. A similar bash script was used to run the program for different configurations and log the outputs.

A similar bash script was used to run the program for different configurations and log the outputs. The performance-critical Matrix-Vector operations were executed **200** times for problem sizes $N = 32, 64$, and 128 . For the largest problem size, $N = \mathbf{256}$, the repeat count was reduced to **50** to manage total execution time.

The results were as follows:

N	Transfer	L2 Error	MV Size	MV Time	MV BW	CG Iters	CG Time	CG Error	MUPD/s/it
32	0.002	0.039	32768	0.000096	75.116	91	0.015	0.0000292	199.063
64	0.016	0.010	262144	0.000739	79.962	174	0.146	0.0000666	312.631
128	0.219	0.00267	2097152	0.00622	77.151	309	2.098	0.000195	308.840
256	1.706	0.00270	16777216	0.0513	75.341	500	27.816	0.000975	301.573

Table 3: CUDA Performance and accuracy results for **float** precision at different domain sizes - CRS Format

N	Transfer	L2 Error	MV Size	MV Time	MV BW	CG Iters	CG Time	CG Error	MUPD/s/it
32	0.00297	0.0395	32768	0.000133	80.736	84	0.0173	5.46e-12	159.474
64	0.0237	0.0102	262144	0.00110	80.296	142	0.193	2.80e-11	192.774
128	0.337	0.00259	2097152	0.00920	78.206	234	2.417	8.78e-11	203.036
256	2.587	0.000652	16777216	0.0761	76.136	332	28.232	3.94e-10	197.297

Table 4: CUDA Performance and accuracy results for **double** precision at different domain sizes - CRS Format

We observe similar results in Tables 3 and Tables 4 with L2 Error decrease with size and the CG Error values range (including the stalling at $N = 256$), and increase in iterations.

The GPU, as expected, exhibits a much higher performance accross ALL the various runs (e.g., about 9-10x speedup on the $N = 256$, `double` case). With the observed conditions, the implementation for SpMVM is most likely **CORRECT** (Although, an observation here is that the bandwidth numbers are much below the theoretical max bandwidth of 300 GB/s as specified in the Nvidia Official specifications [1]).

Task 3: Reading the Kreutzer’s Paper

In the assignment, the following paper is provided for reading to answer the subsequent tasks. The paper[2], authored by Moritz Kreutzer and others, introduced **SELL-C- σ** , a Sliced ELLPACK variant with SIMD vectorization, as a **unified sparse matrix format** optimized for efficient spMVM on CPUs, Xeon Phi, and Nvidia GPUs. CRS is a cache-friendly sparse matrix layout. It uses `val[]` to store all nonzero matrix entries row by row, `col[]` to store the original column indices for each entry in `val[]`, and `rpt[]` to store the starting offsets in `val[]` for each row.

Task 4:

This task comprises of the SELL-C- σ matrix storage format implementation, where the standard CRS format is converted to SELL-C- σ on the host. The implementation builds on the previous CRS implementation’s `sparse_matrix.hpp` file, and uses conditions within the `SparseMatrix` class to switch between the CRS and SELL-C- σ format, requiring new member variables for cell slicing, padding, and a dedicated CUDA kernel (`compute_spmv_cell_c_sigma`).

Additionally another simple program was created, which represents the cuSPARSE implementation of the SpMVM operation (cuSPARSE is implemented with CRS format). This approach utilizes cuSPARSE’s data descriptor objects (`cusparseSpMatDescr_t` and `cusparseDnVecDescr_t`) to define the matrix and vectors on the device before executing the highly tuned routine (`cusparseSpMV`).

The results in Tables 5, 6, 7 and 8, represent the performance and accuracy results of *float* and *double* precision when the SELL-C- σ format is implemented using naive CUDA, and the results of *float* and *double* precision when the SpMVM operation is implemented using cuSPARSE libraries[3], respectively.

N	Transfer	L2 Error	MV Size	MV Time	MV BW	CG Iters	CG Time	CG Error	MUPD/s/it
32	0.360	0.039	32768	5.13e-05	139.685	91	0.013	2.56e-05	233.769
64	0.266	0.010	262144	2.58e-04	229.302	173	0.063	7.57e-05	722.852
128	0.479	0.003	2097152	0.002	239.168	310	0.788	1.96e-04	825.354

Table 5: CUDA Performance and accuracy results for `float` precision at different domain sizes - SELL-C- σ Format.

N	Transfer	L2 Error	MV Size	MV Time	MV BW	CG Iters	CG Time	CG Error	MUPD/s/it
32	0.250	0.039	32768	5.97e-05	180.074	84	0.012	5.46e-12	225.670
64	0.227	0.010	262144	3.72e-04	238.313	142	0.082	2.80e-11	454.185
128	0.550	0.003	2097152	0.0029	248.391	234	0.922	8.78e-11	532.489

Table 6: CUDA Performance and accuracy results for **double** precision at different domain sizes - SELL-C- σ Format.

N	Transfer	L2 Error	MV Size	MV Time	MV BW	CG Iters	CG Time	CG Error	MUPD/s/it
32	0.00250	0.0395	32768	0.000070	100.427	91	0.0137	1.823e-05	216.959
64	0.0161	0.0102	262144	0.000251	231.010	171	0.0626	5.565e-05	716.014
128	0.218	0.00274	2097152	0.00253	185.939	315	0.860	1.752e-04	768.500
256	1.684	0.00365	16777216	0.0189	200.375	500	10.763	1.054e-03	779.413

Table 7: Performance and accuracy results for **float** precision at different domain sizes - cuSPARSE implementation

N	Transfer	L2 Error	MV Size	MV Time	MV BW	CG Iters	CG Time	CG Error	MUPD/s/it
32	0.00305	0.0395	32768	0.000102	104.500	84	0.0153	5.462e-12	180.329
64	0.0242	0.0102	262144	0.000473	185.000	142	0.0911	2.801e-11	408.430
128	0.340	0.00259	2097152	0.00406	175.200	234	1.152	8.783e-11	426.115
256	2.651	0.000652	16777216	0.0349	164.193	333	13.373	3.874e-10	417.769

Table 8: Performance and accuracy results for **double** precision at different domain sizes - cuSPARSE implementation

As we can observe, for both the naive CUDA and cuSPARSE, the L2 Discretization Error decreases, CG Iterations increase, numerical accuracy is within expected range for the corresponding datatype. All of this point out that the implementation is mostly correct.

Performance wise, Float precision achieves a significantly higher value in the MUPD/s/it than double precision (pointed to reduced size and thus higher amount of operations per second). All the time values increase due to usual reasons ($Size \propto Time$). Comparing the storage formats, the SELL-C- σ implementation achieved a massive $\approx 3\times$ speedup in Mat-vec bandwidth (up to around 240 GB/s) and a significant speedup in MUPD/s/it over the simple CRS kernel. The current bandwidth is much more closer to the theoretical GPU bandwidth as well.

But an interesting observation here, is that I wasn't able to generate results for $N = 256$, for the SELL-C- σ case, but it produced results in the CRS case. Since many rows in SELL-C- σ are padded, the actual memory used is greater than the theoretical NNZ size, which increases the memory footprint compared to a clean CRS format. Another interesting observation is that our SELL-C- σ implementation achieved slightly better results accross performance metrics compared to the cuSPARSE implementation. This is likely because our kernel is tailored to the structured sparsity of the FEM matrix, allowing for better memory

coalescing and reduced thread divergence, and so the optimization impact may surpass a built-in library like cuSPARSE.

Task 5: Theoretical Explanations

Insights from the Kreutzer Paper

The primary issue with CRS that SELL-C- σ addresses is that parallelizing CRS on GPUs by assigning one thread per row disrupts memory coalescing. To fix this, one of the other formats we can use is **SELL-C- σ** . It works by dividing the matrix into equally sized “**chunks**” of C rows (Sliced ELLPACK variant). It reduces memory overhead by only padding rows within each chunk to the length of the longest row in that chunk. Crucially, it stores elements in column-major order within the padded chunks, ensuring threads accessing the matrix (one thread per row) hit consecutive memory locations, which is key for memory coalescing. It uses a sorting scope (σ) to group similar-length rows, maximizing efficiency.

Benefits of the SELL-C- σ over CRS

As we know, CRS format stores the matrix row-by-row. But because the lengths of rows are irregular in general sparse matrices, the entries from one row to the next are not uniformly spaced in memory, and thus it causes poor memory coalescing on the GPU. GPUs rely on memory coalescing rather than caching—adjacent threads in a warp must access adjacent memory locations to issue a single wide memory transaction. In contrast, CPUs depend on caching mechanisms that tolerate irregular access relatively better. But in the case of SELL-C- σ , we observe that it achieves a massive speedup in data throughput (about a 2.89x-3.10x speed up in bandwidth), and it is attributable to the theoretical benefits of the SELL-C- σ variant given in the Kreutzer paper. This can be attributed to the fact that the bottlenecked SpMVM operation got optimized. The speedup also translates into the time taken to solve the conjugate gradient, where the SELL-C- σ format achieves approximately a **2.66 $\times$** speedup in CG Solve time for the 128-element float case, which can be attributed to the format advantage of SELL-C- σ .

Task 6:

For this task, an appropriate methodology would be studying the transfer times and CG solve times in CRS and SELL-C- σ formats for one example, such as `float`.

CRS format (`float`):

N	Transfer Time (s)	CG Solve Time (s)	Total Time (s)
32	0.002	0.015	0.017
64	0.016	0.146	0.162
128	0.219	2.098	2.317

SELL-C- σ format (`float`):

N	Conversion Time (s)	Transfer Time (s)	CG Solve Time (s)	Total Time (s)
32	0.058	0.360	0.013	0.431
64	0.36	0.266	0.063	0.689
128	2.88	0.479	0.768	4.127

The matrix-vector bandwidth (MV BW) is consistently higher in the SELL-C- σ format across every measured size compared to the CRS version.

As we observe, the SELL-C- σ is transferred substantially slower than the **CRS** format, indicating the significance of the padding overhead. Similarly, the CG Solve Time, especially as the matrix grows larger, is much faster (or less) in the SELL-C- σ format, due to the discussed advantages of the latter in Task 5, again proving that the SELL-C- σ format is superior for faster compute of CG.

An additional experiment was tried while reporting this task. The conversion from CRS to SELL-C- σ format in the CPU was recorded, and added to the above table. Looking at the results, a large part of the SELL-C- σ operation runtime was spent on the format conversion. Upon obtaining the total time, the overall time actually becomes less than the CRS case, which seems counterintuitive. Looking at the results, a large part of the SELL-C- σ operation runtime was spent on the format conversion and the Host-to-Device transfer of the padded structure. Upon obtaining the total time, the overall time is now significantly higher than the CRS, which makes it seem counterintuitive to use SELL-C- σ .

However, we can mitigate this using two potential ideas:

- The conversion-transfer operation can be treated as a fixed overhead and can be ‘amortized’ over multiple solves, i.e., if you assume that a large number of SpMVM operations will be executed in sequence, the resultant computational performance benefit is high that the initial cost becomes negligible relative to the total time saved across all repetitions.
- The slow conversion operation is currently CPU-bound. Moving the entire CRS-to-SELL-C- σ conversion process to the GPU, but it has to be realized through practical implementation to observe the true parallelism.

Task 7:

For this task, the subtasks would be to think about extent of possibility upto which these frameworks could be used to implement the two discussed formats (CRS and SELL-C- σ), and how would it be implemented, if possible.

For the CRS Format, both the Kokkos and cuSPARSE frameworks can be readily used to implement the SpMVM operation. Using `cusparsespmv` in cuSPARSE, we were able to provide a high-level implementation in the CRS format, which was represented by tables 7 and 8 in task 4. Similarly, Kokkos provides a class named `CrsMatrix`[4] for implementing Sparse Matrices directly in CRS format, and a function `KokkosSparse::spmv`[5] for multiplying matrices in CRS or CSR formats.

For the SELL-C- σ format, the Kokkos doesn't directly support it, although it is possible to implement a custom kernel which at the same time can also ensure portability using Kokkos features like `Kokkos::View` (which can be used to implement multi-dimensional arrays) and `Kokkos::parallel_for`, and retaining the performance optimizations given by the SELL-C- σ format. The only consideration is that, since the custom flattened SELL-C- σ layout is highly tuned for 1D memory coalescing, it does not align naturally with the standard multi-dimensional Kokkos::View abstraction; hence, we must use views carefully (often preferring 1D Views) and pay attention to memory access patterns to preserve performance gains. The cuSPARSE framework is NOT suitable for implementing SELL-C- σ because of its highly specialized nature that it can support only a few formats (CRS being one of them, while SELL-C- σ isn't supported).

In our code, since the main optimization focuses on the implementation of SELL-C- σ , Kokkos could be a preferred choice. However, as a point of comparison against a custom developed CUDA kernel (SELL-C- σ in this case), the cuSPARSE library is easier to implement.

Points to Acknowledge:

The following sources were used in the implementation of my project.

1. For the technical content, the lecture notes[6] are referred.
2. **Google AI:** Google AI tools, Gemini[7] And NotebookLM[8] were used to generate quick boilerplates and extracting interesting details in developer docs (APIs used in latest versions). Also used to polish report writing, although minimally.
3. Other docs cited in appropriate places

References

- [1] Nvidia tesla t4 tensor core gpu datasheet. <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet-951643.pdf>, 2019.
- [2] Moritz Kreutzer, Georg Hager, Gerhard Wellein, Holger Fehske, and Alan R. Bishop. A unified sparse matrix data format for efficient general sparse matrix-vector multiplication on modern processors with wide simd units. *SIAM Journal on Scientific Computing*, 36(5):C401–C423, 2014.
- [3] NVIDIA Corporation. cusparse 13.0 — using the cusparse api. <https://docs.nvidia.com/cuda/cusparse/#using-the-cusparse-api>, 2025.
- [4] Kokkos Team. Kokkosparse::crsmatrix — kokkos kernels documentation. https://kokkos.org/kokkos-kernels/docs/API/sparse/crs_matrix.html, 2025.
- [5] Kokkos Team. Kokkosparse::spmv — kokkos kernels documentation. <https://kokkos.org/kokkos-kernels/docs/API/sparse/spmv.html>, 2025.

- [6] Uppsala University. Studium modules on “acceleration-based programming”. <https://uppsala.instructure.com/courses/107322/modules>.
- [7] Google DeepMind. Gemini. <https://gemini.google.com/app>, 2025.
- [8] Google Labs. Notebooklm. <https://notebooklm.google/>, 2023.